

Les deux tests auditifs de HearSafe

Fonctionnement technique & production par le code

Test tonal pur (PTT) · Test de fréquence maximale audible (HFRT)

Application mobile React Native / Expo · TypeScript
Génération du son : WebView cachée + Web Audio API

1. Vue d'ensemble & principe commun

L'application propose **deux tests auditifs** complémentaires :

Test	Ce qu'il mesure	Axe exploré	Plage
PTT — test tonal pur (audiométrie tonale liminaire)	Le seuil auditif : le son le plus faible perçu, pour chaque fréquence	Intensité (dB)	500, 1000, 2000, 4000 Hz
HFRT — fréquence max audible (« mosquito tone »)	La fréquence la plus aiguë encore perçue	Fréquence (Hz)	8 kHz → 20 kHz

Le principe est le même pour les deux — c'est celui de l'audiométrie clinique (Hughson-Westlake / Békésy), le HFRT n'étant qu'une **transposition** du PTT sur l'axe des fréquences :

- **Escalier adaptatif** : l'utilisateur maintient un bouton tant qu'il entend, le relâche sinon.
- **Deux phases** : recherche *grossière* (gros pas, rapide) puis suivi *fin* (petits pas, précis).
- **Inversions (basculements)** : chaque changement *maintenir*↔*relâcher* encadre la limite.
- **Moyenne des inversions** (la 1^{re} écartée) = la valeur mesurée précise.

Cadre honnête : matériel non calibré, résultats **non cliniques**. Les dB du PTT sont une atténuation logicielle relative (pas du dB HL) ; le HFRT dépend fortement des écouteurs. L'objectif est la **sensibilisation**, pas le diagnostic.

2. Production du son — commune aux deux tests

React Native n'a pas de synthétiseur audio natif. La solution : une **WebView cachée** (`react-native-webview`) qui exécute l'**API Web Audio** du navigateur. Le son pur (sinusoïde) y est généré par un oscillateur.

2.1 Le générateur (dans la WebView) — `hearing-test.constants.ts`

```
// WEBVIEW_AUDIO_HTML - fonction playTone()
var osc = ctx.createOscillator();
osc.type = 'sine'; // onde SINUSOÏDALE (son pur)
osc.frequency.value = frequency; // hauteur en Hz
gain.gain.linearRampToValueAtTime(volume, ctx.currentTime + 0.025); // fondu anti-clac
gain.connect(ctx.destination); // vers le haut-parleur
osc.start(); // ▶ le son sort

window.setFrequency = function(f){ _osc.frequency.setValueAtTime(f, _ctx.currentTime); }; // glissando
```

2.2 Le pont React Native ↔ WebView — `audio/AudioEngine.tsx`

Le composant expose `playTone`, `setFrequency`, `stopTone`. Ces appels sont injectés dans la WebView :

```
setFrequency(f){ webViewRef.current?.injectJavaScript(`window.setFrequency && window.setFrequency(${f});`); }
```

Piège connu & corrigé : l'injection fait `window.fn && window.fn( ... )`. Si une fonction manque côté WebView, l'appel est un **no-op silencieux** (aucune erreur). C'est ce qui avait rendu le HFRT muet un temps : `setFrequency` avait disparu du HTML → le son restait figé.

3. Test tonal pur (PTT)

Fichiers : `PTTAlgorithm.ts`, `usePureToneTest.ts`, `AudiogramChart.tsx`, `PTTResultView.tsx`.

3.1 D roul 

Pour chaque oreille, on teste 4 fr quences en s quence. Le son est **puls  (250 ms ON / 350 ms OFF)**, cycle de 600 ms = tick de d cision). L'utilisateur **maintient** « J'entends » tant qu'il per oit le bip, **rel che** sinon.

3.2 L'escalier   deux phases (Hughson-Westlake)

	Phase 1 — Recherche	Phase 2 — Suivi
D�clencheur	0 inversion	� 1 inversion
Pas (asym�triques / sym�triques)	−10 dB si entendu / +5 dB sinon	�3 dB
But	Cadrer vite la zone du seuil	Osciller finement autour du seuil

C'est la r gle clinique « **10 down / 5 up** » : on descend fort d s qu'on entend (pour passer sous le seuil), on remonte doucement sinon.

Le pas, dans le code — `PTTAlgorithm.ts`

```
export function adjustDb(currentDb, isHeld, reversalCount) {
  const steps = getStepsForPhase(reversalCount); // 0 inv. → -10/+5 ; sinon →  3
  const next = isHeld ? currentDb - steps.hold : currentDb + steps.release;
  return Math.min(PTT_MAX_DB, Math.max(PTT_MIN_DB, next)); // born  [-20, 80]
}
```

3.3 Les inversions et la formule du seuil

Une **inversion** est un changement de sens *maintenir*→*rel cher*. On vise **6 inversions**. La 1 e (issu  de la phase grossi re, biais e par le point de d part) est ** cart e**, puis on **moyenne** les dB des inversions restantes :

$$\text{seuil} = (1 / (n-1)) \cdot \sum_{i=2..n} \text{dB}_i$$

```
export function computeThresholdDb(reversals) {
  const usable = reversals.length   2 ? reversals.slice(1) : reversals; // on jette la 1 re
  const sum = usable.reduce((acc, r) => acc + r.db, 0);
  return Math.round(sum / usable.length); // moyenne arithm tique (axe dB)
}
```

3.4 Du dB au son : la conversion

Le dB interne est transform  en gain Web Audio (formule inverse du d cibel, r f rence 60) :

$$\text{volume} = \min(1, \max(0.00005, 10^{(\text{dB} - 60) / 20}))$$

3.5 Boucle temps r el — `usePureToneTest.ts`

L' tat de l'algorithme vit dans des `useRef` (et non `useState`) : la boucle tourne sur un timer de 600 ms ; un `ref` garantit que le callback lit toujours la valeur **fra che et synchrone** (pas une closure p rim e). Chaque cycle : on  chantillonne l'appui → on d tecte l'inversion → on ajuste le dB → on joue le bip 250 ms → on planifie le suivant.

3.6 Résultat : l'audiogramme

La moyenne des 4 seuils par oreille donne le **PTA-4** (méthode OMS). L'**audiogramme** ([AudiogramChart.tsx](#), dessiné en `View` sans librairie SVG) place :

- **Axe X** = fréquence en échelle **logarithmique** : $x = (\ln f - \ln f_{\min}) / (\ln f_{\max} - \ln f_{\min})$
- **Axe Y** = dB en échelle linéaire **inversée** (haut = meilleure audition — convention clinique)
- Oreille droite = cercle rouge (O), gauche = croix bleue (X)

4. Test de fréquence maximale audible (HFRT)

Fichiers : `HFRTAlgorithm.ts`, `useHighFrequencyTest.ts`, `HFRTResultView.tsx`, `HFRTSpectrumChart.tsx`.

Le HFRT reprend **exactement la logique du PTT** (escalier, inversions, moyenne), mais transposée à l'axe des **fréquences**. Le pas devient **multiplicatif** (logarithmique), car la hauteur tonale se perçoit de façon logarithmique.

4.1 La mécanique : escalier piloté par l'appui

Action	Effet sur la fréquence
Maintenir « J'entends » (tu perçois le son)	elle MONTE (vers 20 kHz)
Relâcher (tu n'entends plus)	elle DESCEND (vers 8 kHz)

Le son **démarre à 8 kHz**. Tant que tu l'entends tu maintiens (ça monte) ; dès qu'il devient inaudible tu relâches (ça redescend) → ça oscille autour de ta limite.

4.2 Les deux phases

	Phase 1 — Recherche (grossière)	Phase 2 — Suivi (fine)
Déclencheur	tant que le son n'a jamais été perdu en montant	dès le 1 ^{er} son perdu en montant (<code>everLostTone</code>)
Pas (par 100 ms)	$\times 1.018 / \times 0.982$ ($\pm 1,8\%$)	$\times 1.012 / \times 0.988$ ($\pm 1,2\%$)
But	rejoindre vite la zone de la limite	préciser la limite

Le pas, dans le code — `HFRTAlgorithm.ts`

```
export function adjustStaircaseFrequency(currentFreq, isHeld, isFine) {
  const up   = isFine ? HFRT_TRACK_UP   : HFRT_SEARCH_UP;   // 1.012 : 1.018
  const down = isFine ? HFRT_TRACK_DOWN : HFRT_SEARCH_DOWN; // 0.988 : 0.982
  const next = isHeld ? currentFreq * up : currentFreq * down;
  return Math.min(HFRT_MAX_FREQ, Math.max(HFRT_MIN_FREQ, next)); // borné [8k, 20k]
}
```

4.3 Les inversions et la formule (transposée du PTT)

Même comptage d'inversions que le PTT (on vise **8 inversions**). La formule est la moyenne des inversions **en domaine logarithmique** = **moyenne géométrique** des fréquences (la 1^{re} écartée) :

$$f_{\max} = \exp\left(\left(1 / (n-1)\right) \cdot \sum_{i=2..n} \ln(f_i)\right)$$

```
export function computeRefinedFrequency(reversals) {
  const usable = reversals.length ≥ 2 ? reversals.slice(1) : reversals; // on jette la 1ère
  const logSum = usable.reduce((acc, f) => acc + Math.log(f), 0);
  return Math.round(Math.exp(logSum / usable.length)); // moyenne GÉOMÉTRIQUE
}
```

Géométrique et non arithmétique : moyenner en Hz bruts surpondérerait les inversions hautes ; en `ln(f)` chaque octave pèse pareil, ce qui correspond à la perception.

4.4 Détection des inversions — HFRTAlgorithm.ts

```
export function recordReversal(reversals, lastTransition, newTransition, currentFreq) {
  if (lastTransition === null || lastTransition === newTransition)
    return { reversals, isReversal: false }; // même sens → pas d'inversion
  return { reversals: [...reversals, Math.round(currentFreq)], isReversal: true }; // basculement !
}
```

4.5 La boucle — useHighFrequencyTest.ts

Un tick toutes les 100 ms (état dans des `useRef`, comme le PTT). À chaque tick : échantillonner l'appui → détecter l'inversion → basculer en pas fins au 1^{er} son perdu → ajuster la fréquence → mettre à jour le son via `setFrequency`. Conditions de fin : **8 inversions** (→ formule), plafond 20 kHz atteint, blocage (aucune inversion pendant 10 s), ou « aucune réponse ».

4.6 Interprétation selon l'âge

La fréquence max chute avec l'âge (presbycousie). Une heuristique compare le résultat à la moyenne attendue pour l'âge (via la date de naissance du profil) : « *au-dessus / typique / en-dessous de votre âge* ». Modèle documenté comme **indicatif, non diagnostique**.

5. Le parallèle PTT ↔ HFRT

C'est le même algorithme, appliqué à deux axes différents :

Élément	PTT (test tonal)	HFRT (haute fréquence)
Ce qu'on cherche	seuil d'intensité	fréquence limite
Axe	Intensité — dB	Fréquence — Hz
Nature du pas	additif (±dB)	multiplicatif / log (×facteur)
Signal	pulsé (250/350 ms)	continu (glissando)
Phase grossière	− 10 / +5 dB	× 1.018 / × 0.982
Phase fine	± 3 dB	× 1.012 / × 0.988
Inversions cibles	6	8
Formule (1 ^{re} écartée)	moyenne arithmétique des dB	moyenne géométrique des Hz
Restitution	audiogramme (PTA-4)	échelle 8–20 kHz + zones d'âge

Phrase de synthèse (soutenance) : « Les deux tests reposent sur l'escalier adaptatif de Hughson-Westlake : recherche grossière puis suivi fin, avec comptage des inversions et moyenne (la première écartée). Le PTT le fait sur l'intensité (dB, pas additifs, moyenne arithmétique) pour trouver le seuil ; le HFRT le transpose à la fréquence (Hz, pas multiplicatifs, moyenne géométrique) pour trouver la limite haute. Le son est généré par un oscillateur Web Audio dans une WebView, piloté depuis React Native. »

6. Limites assumées

- Matériel **non calibré** : les résultats ne sont **pas cliniques** (pas de dB HL).
- Le HFRT dépend fortement des écouteurs et de l'environnement.
- Le timing des pulses/pas est piloté côté JS (non garanti à la ms près).
- Objectif = **sensibilisation**, avec renvoi vers un audiologiste pour tout diagnostic.